

NP-Completeness and Reduction Techniques

How Efficient Transformations Prove Computational Hardness

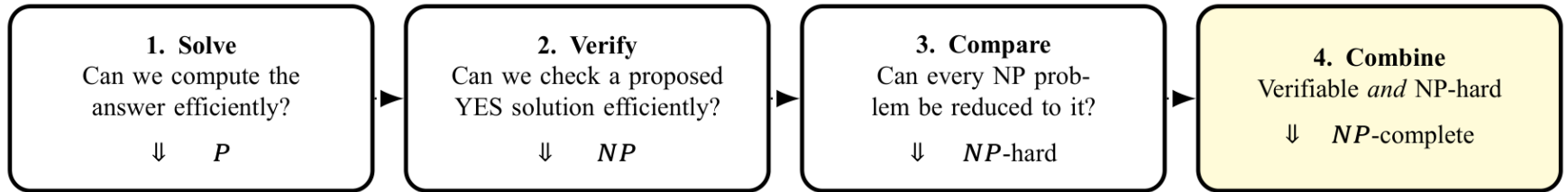
Most. Sonia Khatun

BSc in Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology (BUET)

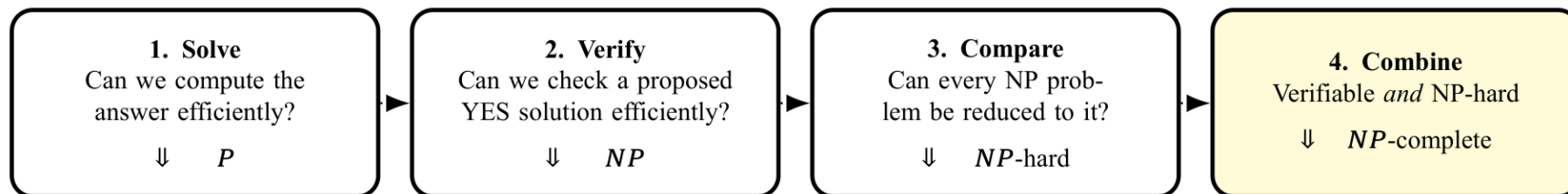
Demo Lecture for the Position of Lecturer

Department of Computer Science and Engineering
Green University of Bangladesh

Why Do We Need NP-Completeness?



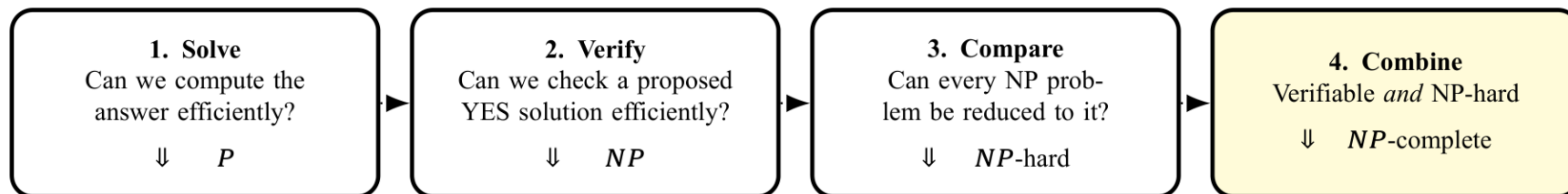
Why Do We Need NP-Completeness?



The central proof question

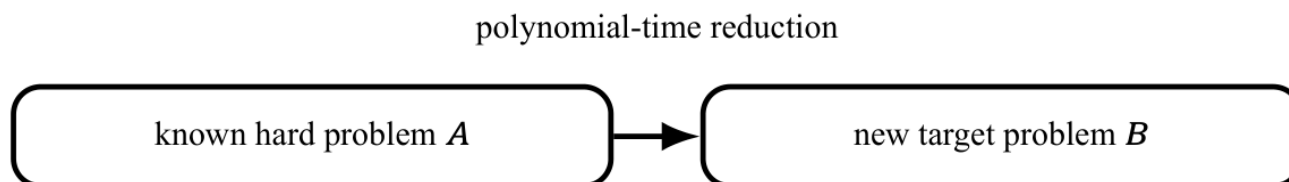
How can we transfer the known hardness of one problem to a new problem?

Why Do We Need NP-Completeness?



The central proof question

How can we transfer the known hardness of one problem to a new problem?

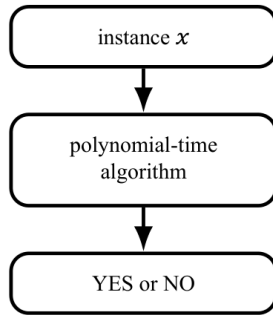


The dotted arrows show the learning sequence, not set containment.

Start with P and NP

P: solve efficiently

Decision problems solvable by a deterministic algorithm in polynomial time.

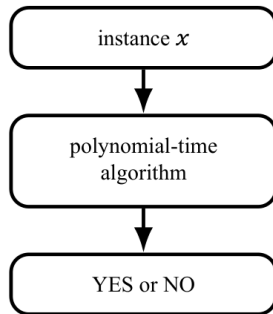


Example: reachability can be solved by BFS or DFS.

Start with P and NP

P: solve efficiently

Decision problems solvable by a deterministic algorithm in polynomial time.

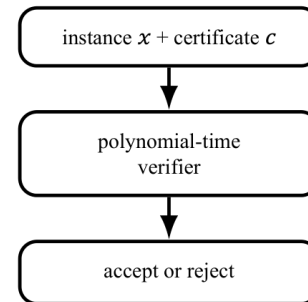


Example: reachability can be solved by BFS or DFS.

NP: verify efficiently

There are a polynomial-time verifier V and polynomial p such that

$$x \in L \Leftrightarrow \exists c: \begin{array}{l} |c| \leq p(|x|), \\ V(x, c) = 1. \end{array}$$

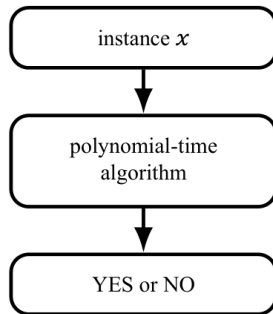


For 3-COLOR, c assigns one of three colors to every vertex; check every edge.

Start with P and NP

P: solve efficiently

Decision problems solvable by a deterministic algorithm in polynomial time.



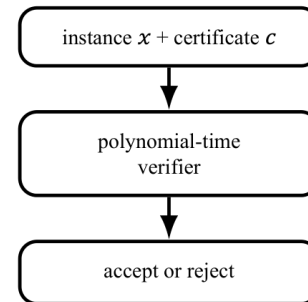
Example: reachability can be solved by BFS or DFS.

$P \subseteq NP$: a polynomial-time solver can serve as a verifier and ignore the certificate.

NP: verify efficiently

There are a polynomial-time verifier V and polynomial p such that

$$x \in L \Leftrightarrow \exists c: \begin{array}{l} |c| \leq p(|x|), \\ V(x, c) = 1. \end{array}$$



For 3-COLOR, c assigns one of three colors to every vertex; check every edge.

NP means **nondeterministic polynomial time**, not “non-polynomial.” Whether $P = NP$ is open.

From NP-Hard to NP-Complete

NP-hard: hard for all of NP

A problem H is NP-hard if

$$\forall L \in NP, \quad L \leq_p H.$$

$L \leq_p H$ means an efficient, answer-preserving translation from L to H . An NP-hard problem need not itself belong to NP.

NP-complete: both properties

$$H \in NP \quad \text{and} \quad H \text{ is NP-hard.}$$

These are the hardest efficiently verifiable decision problems.

From NP-Hard to NP-Complete

NP-hard: hard for all of NP

A problem H is NP-hard if

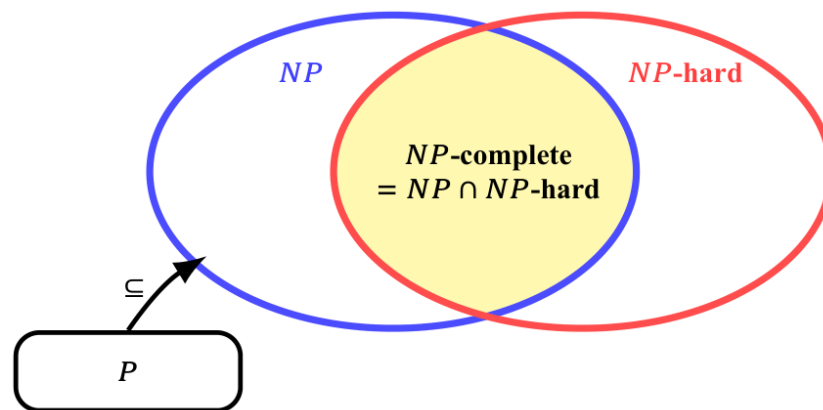
$$\forall L \in NP, \quad L \leq_p H.$$

$L \leq_p H$ means an efficient, answer-preserving translation from L to H . An NP-hard problem need not itself belong to NP.

NP-complete: both properties

$$H \in NP \quad \text{and} \quad H \text{ is NP-hard.}$$

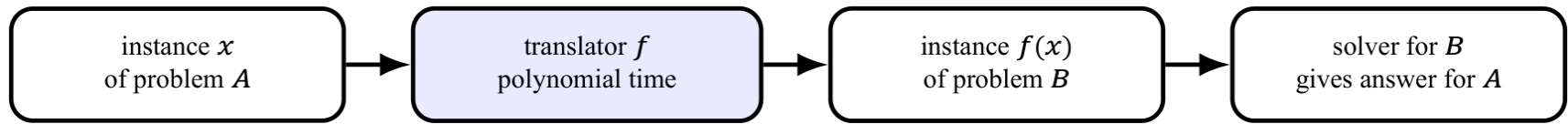
These are the hardest efficiently verifiable decision problems.



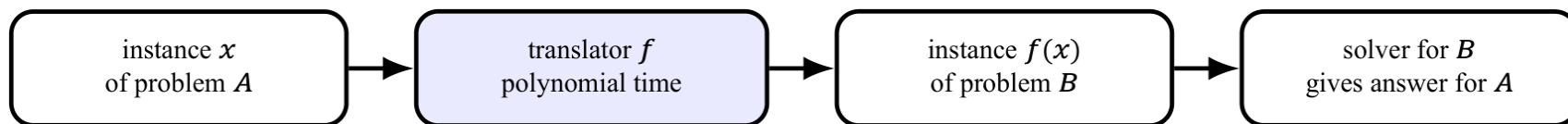
$P \subseteq NP$ is known. The exact boundary of P is not drawn because whether $P = NP$ is open.

If one NP-complete problem is in P , then $P = NP$.

Polynomial-Time Reduction: an Answer-Preserving Translator



Polynomial-Time Reduction: an Answer-Preserving Translator



Formal meaning

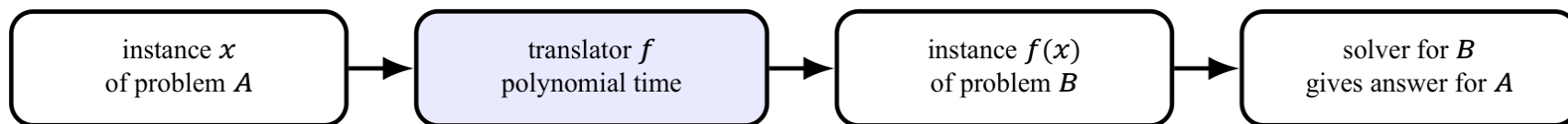
$A \leq_p B$: there exists a polynomial-time computable function f such that

$$\forall x, \quad x \in A \Leftrightarrow f(x) \in B.$$

A solver for B would also solve A : B is at least as hard as A .

To prove B hard: known hard $A \rightarrow$ target B .

Polynomial-Time Reduction: an Answer-Preserving Translator



Formal meaning

$A \leq_p B$: there exists a polynomial-time computable function f such that

$$\forall x, \quad x \in A \Leftrightarrow f(x) \in B.$$

A solver for B would also solve A : B is at least as hard as A .

To prove B hard: known hard $A \rightarrow$ target B .

Tiny example: EVEN reduces to MULTIPLE-OF-4

Use $f(n) = 2n$. Then

$$n \text{ is even} \Leftrightarrow 4 \mid 2n.$$

n	f(n)	Preserved?
6: YES	12: YES	YES
7: NO	14: NO	YES

This illustrates the mechanics; a hardness proof starts from a known hard problem.

Reusable Recipe: Proving a Problem NP-Complete

1. Membership in NP

State a polynomial-size certificate.

Give a polynomial-time verifier.

$$B \in NP.$$

Reusable Recipe: Proving a Problem NP-Complete

1. Membership in NP

State a polynomial-size certificate.

Give a polynomial-time verifier.

$$B \in NP.$$

2. NP-hardness

1. Choose a known NP-complete problem A .
2. Construct f in polynomial time.
3. Prove $x \in A \Leftrightarrow f(x) \in B$.

If A is NP-hard and $A \leq_p B$, then B is NP-hard.

Reusable Recipe: Proving a Problem NP-Complete

1. Membership in NP

State a polynomial-size certificate.

Give a polynomial-time verifier.

$$B \in NP.$$

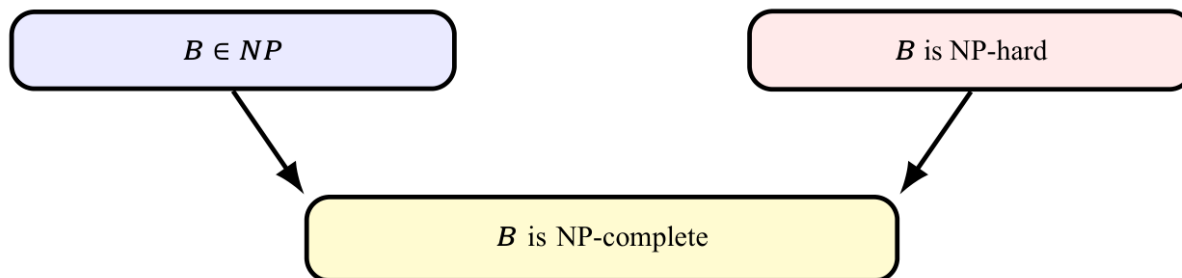
2. NP-hardness

1. Choose a known NP-complete problem A .

2. Construct f in polynomial time.

3. Prove $x \in A \Leftrightarrow f(x) \in B$.

If A is NP-hard and $A \leq_p B$, then B is NP-hard.



Direction mnemonic: the known hard problem goes *into* the target problem.

Worked Reduction: 3-SAT reduces to 3-COLOR

Source: 3-SAT

A 3-CNF formula is an AND of clauses, each an OR of three literals:

$$\phi = C_1 \wedge \cdots \wedge C_m.$$

Can a truth assignment satisfy every clause?

A literal is a variable or its negation.

Target: 3-COLOR

Can the graph's vertices be colored with three colors so that adjacent vertices have different colors?

Worked Reduction: 3-SAT reduces to 3-COLOR

Source: 3-SAT

A 3-CNF formula is an AND of clauses, each an OR of three literals:

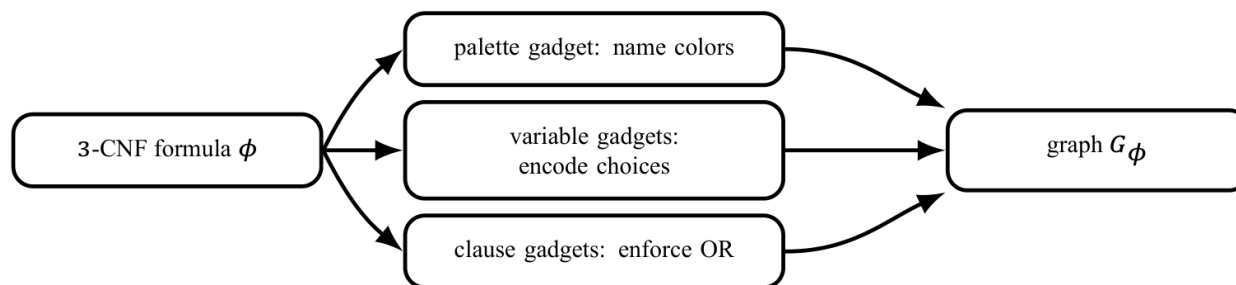
$$\phi = C_1 \wedge \cdots \wedge C_m.$$

Can a truth assignment satisfy every clause?

A literal is a variable or its negation.

Target: 3-COLOR

Can the graph's vertices be colored with three colors so that adjacent vertices have different colors?



Worked Reduction: 3-SAT reduces to 3-COLOR

Source: 3-SAT

A 3-CNF formula is an AND of clauses, each an OR of three literals:

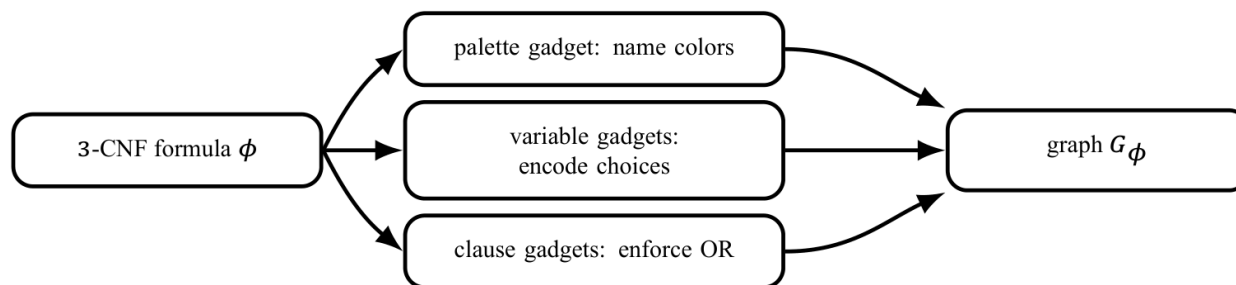
$$\phi = C_1 \wedge \cdots \wedge C_m.$$

Can a truth assignment satisfy every clause?

A literal is a variable or its negation.

Target: 3-COLOR

Can the graph's vertices be colored with three colors so that adjacent vertices have different colors?



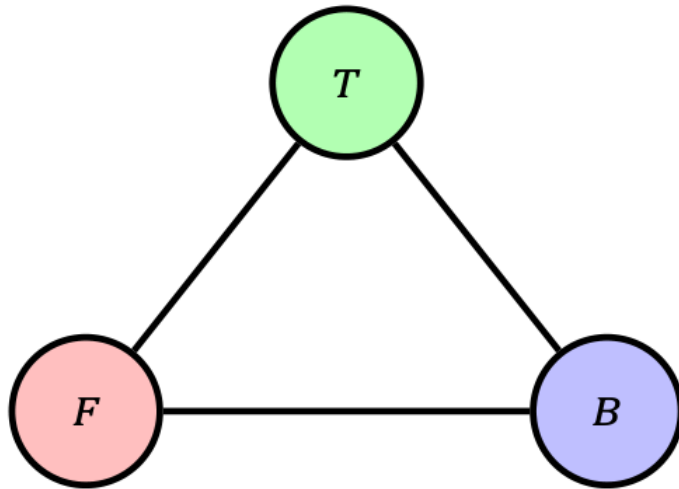
Design invariant: ϕ is satisfiable if and only if G_ϕ is 3-colorable.

This illustrates the general proof recipe; it is not the definition of reduction.

Encoding Truth with Colors

Palette gadget

A triangle forces three distinct colors.
Name them T , F and B (base/helper).

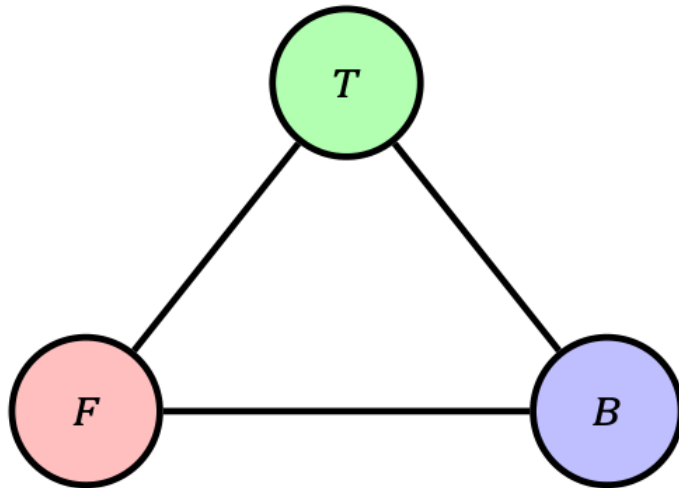


three vertices $\Rightarrow$ three meanings

Encoding Truth with Colors

Palette gadget

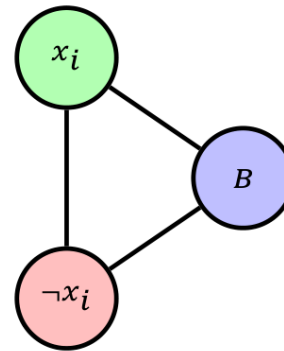
A triangle forces three distinct colors.
Name them T , F and B (base/helper).



three vertices $\Rightarrow$ three meanings

Variable gadget

Connect x_i and $\neg x_i$ to each other and to B .



Exactly two logical colorings:

$$(x_i, \neg x_i) = (T, F)$$

or

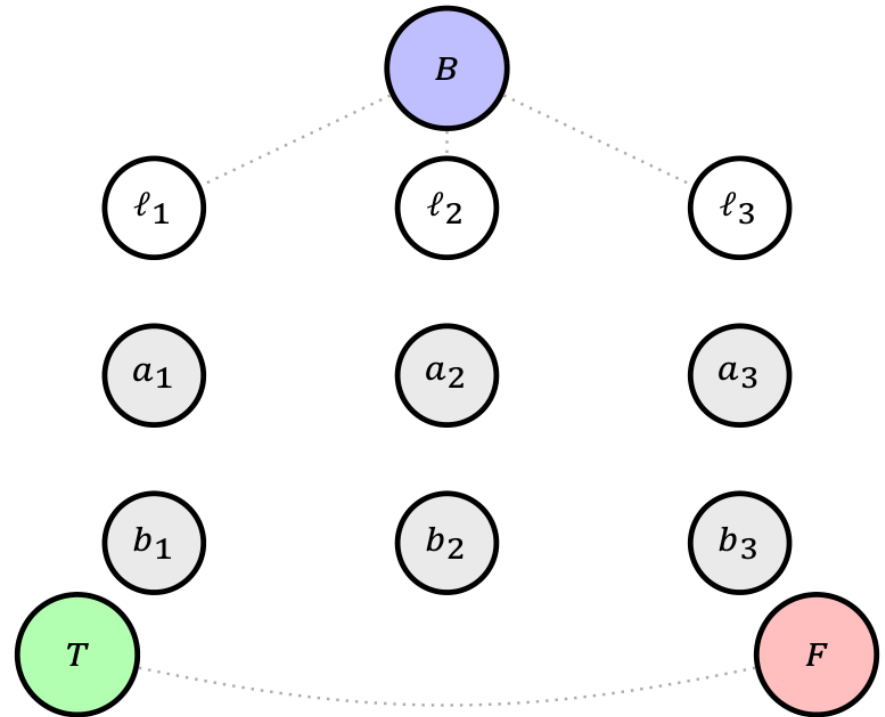
$$(x_i, \neg x_i) = (F, T).$$

Exactly one of x_i and $\neg x_i$ receives color T .

Constructing the Exact Clause Gadget

For $C = (\ell_1 \vee \ell_2 \vee \ell_3)$, add six fresh vertices a_i, b_i and 13 edges:

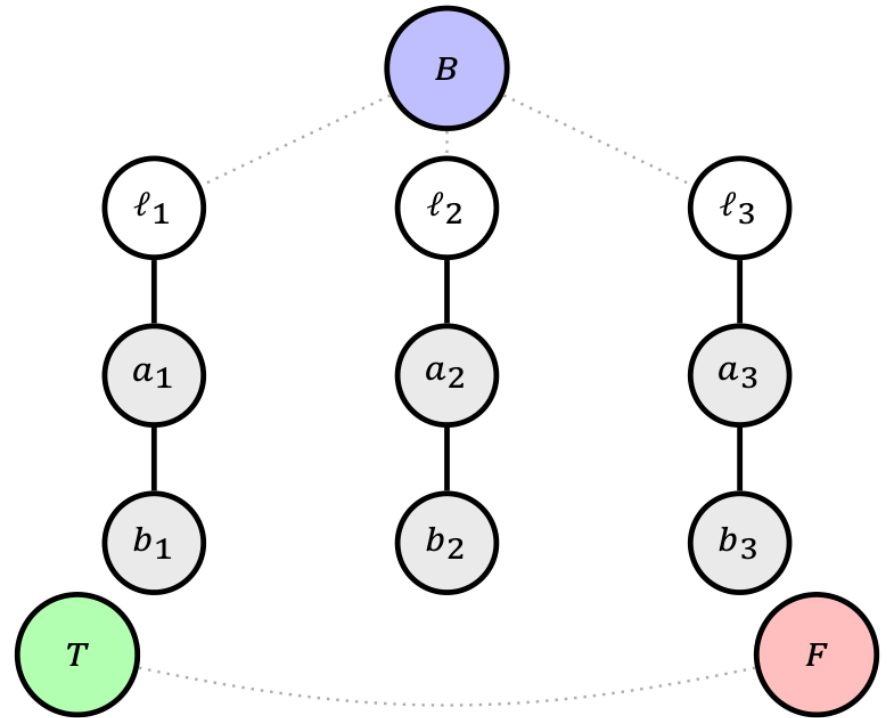
$(\ell_i, a_i), (a_i, b_i), (T, a_i)$
for $i = 1, 2, 3$,
 $(T, b_1), (b_1, b_2),$
 $(b_2, b_3), (b_3, F).$



Constructing the Exact Clause Gadget

For $C = (\ell_1 \vee \ell_2 \vee \ell_3)$, add six fresh vertices a_i, b_i and 13 edges:

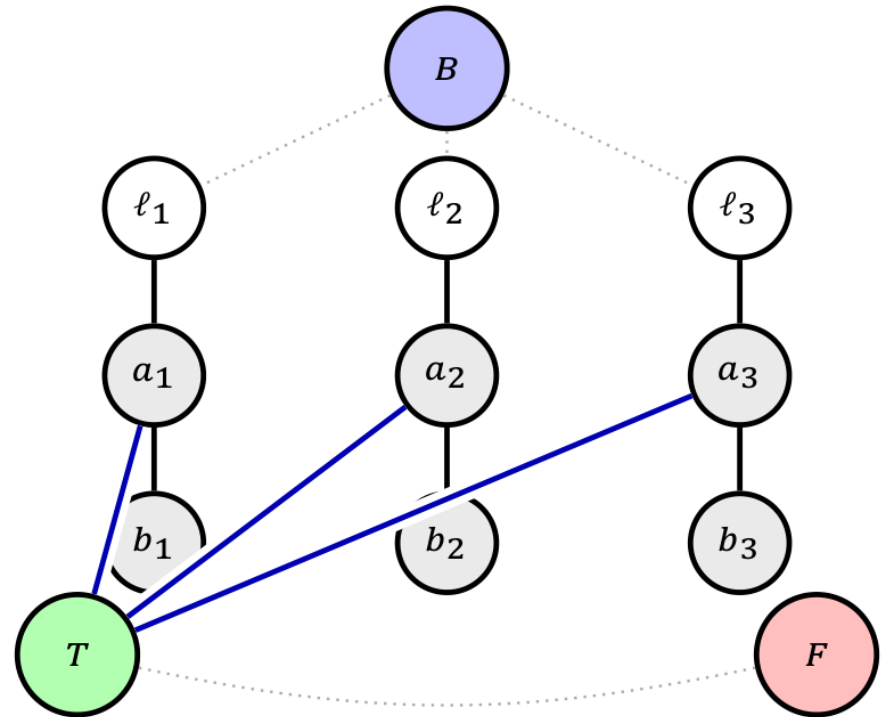
$(\ell_i, a_i), (a_i, b_i), (T, a_i)$
for $i = 1, 2, 3$,
 $(T, b_1), (b_1, b_2),$
 $(b_2, b_3), (b_3, F).$



Constructing the Exact Clause Gadget

For $C = (\ell_1 \vee \ell_2 \vee \ell_3)$, add six fresh vertices a_i, b_i and 13 edges:

$(\ell_i, a_i), (a_i, b_i), (T, a_i)$
for $i = 1, 2, 3$,
 $(T, b_1), (b_1, b_2),$
 $(b_2, b_3), (b_3, F).$



Constructing the Exact Clause Gadget

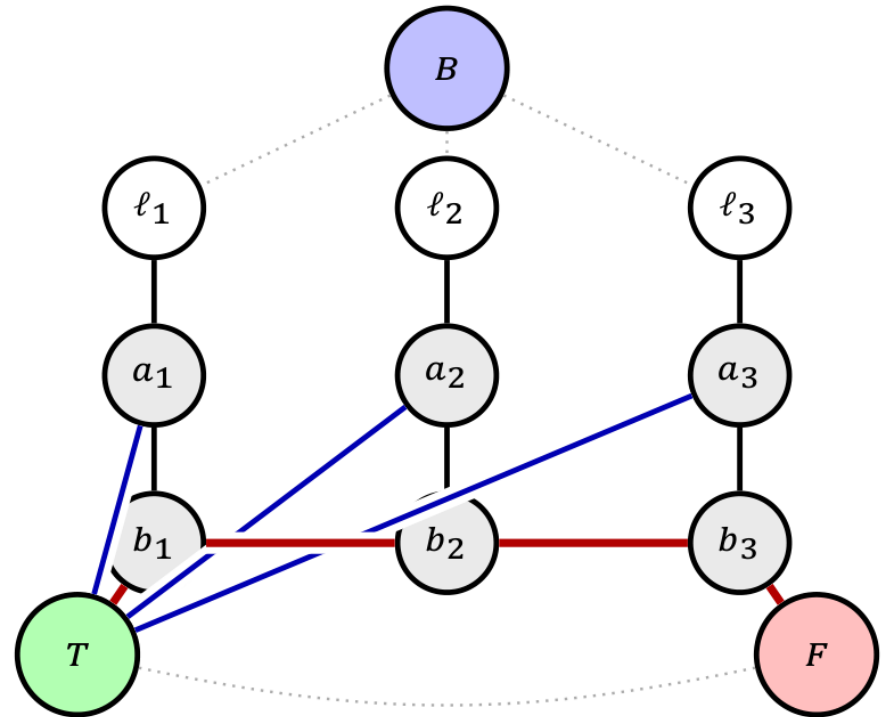
For $C = (\ell_1 \vee \ell_2 \vee \ell_3)$, add six fresh vertices a_i, b_i and 13 edges:

$$\begin{aligned} &(\ell_i, a_i), (a_i, b_i), (T, a_i) \\ &\quad \text{for } i = 1, 2, 3, \\ &(T, b_1), (b_1, b_2), \\ &(b_2, b_3), (b_3, F). \end{aligned}$$

Result: can we complete the coloring?

With inputs fixed in $\{T, F\}$, the internal coloring extends exactly when at least one input is T .

There is no output vertex.



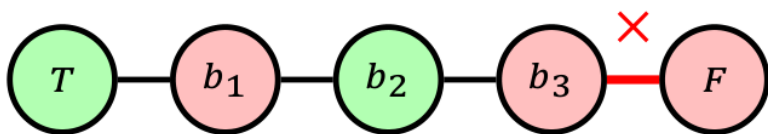
Black: literal arms. Blue: T links. Red: forcing path.
Dotted gray: existing edges.

All lines are ordinary edges. White/gray vertices are unassigned, not extra colors.

Why the Clause Gadget Encodes OR

All inputs are F : impossible

Each a_i is forced to B . Each b_i can therefore use only T or F .

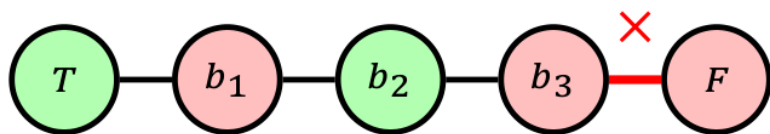


From T , the first three edges force F, T, F .
The last edge then joins two F vertices.

Why the Clause Gadget Encodes OR

All inputs are F : impossible

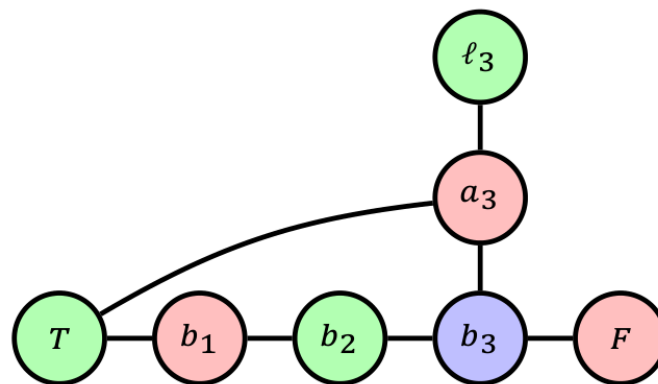
Each a_i is forced to B . Each b_i can therefore use only T or F .



From T , the first three edges force F, T, F .
The last edge then joins two F vertices.

At least one input is T : possible

Choose one true input ℓ_i . Set $a_i = F$, $b_i = B$ and every other $a_j = B$.

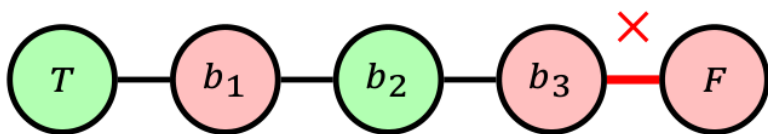


Chosen true position	(b_1, b_2, b_3)
$i = 1$	(B, F, T)
$i = 2$	(F, B, T)
$i = 3$ (shown)	(F, T, B)

Why the Clause Gadget Encodes OR

All inputs are F : impossible

Each a_i is forced to B . Each b_i can therefore use only T or F .



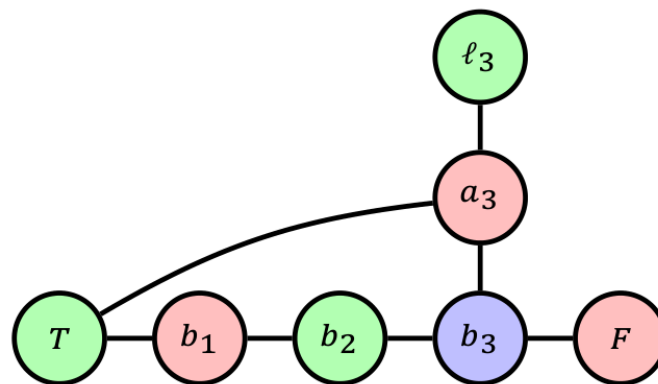
From T , the first three edges force F, T, F . The last edge then joins two F vertices.

F, F, F : no extension.

All seven other input patterns have an extension.

At least one input is T : possible

Choose one true input ℓ_i . Set $a_i = F$, $b_i = B$ and every other $a_j = B$.



Chosen true position	(b_1, b_2, b_3)
$i = 1$	(B, F, T)
$i = 2$	(F, B, T)
$i = 3$ (shown)	(F, T, B)

Evaluating One Complete Formula

$$\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3).$$

Evaluating One Complete Formula

$$\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3).$$

Clause input ports

$$C_1: (x_1, \neg x_2, x_3),$$

$$C_2: (\neg x_1, x_2, \neg x_3).$$

Each clause receives its own six fresh internal vertices.

Evaluating One Complete Formula

$$\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3).$$

Clause input ports

$$C_1: (x_1, \neg x_2, x_3),$$

$$C_2: (\neg x_1, x_2, \neg x_3).$$

Each clause receives its own six fresh internal vertices.

One satisfying assignment

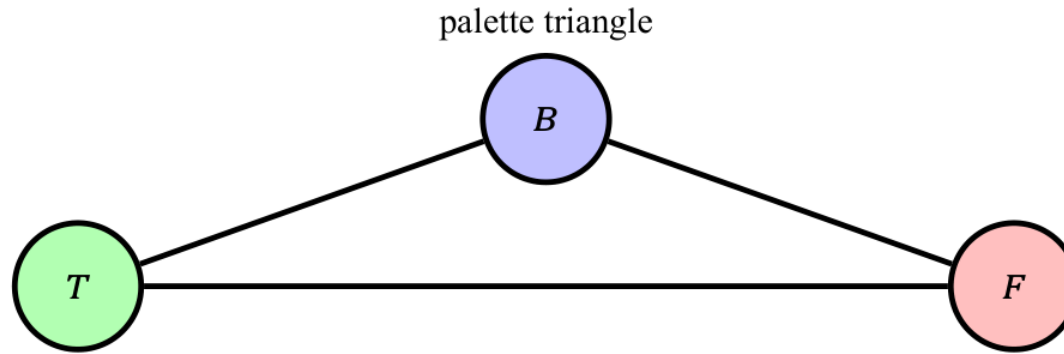
$$x_1 = T, \quad x_2 = T, \quad x_3 = F.$$

Thus

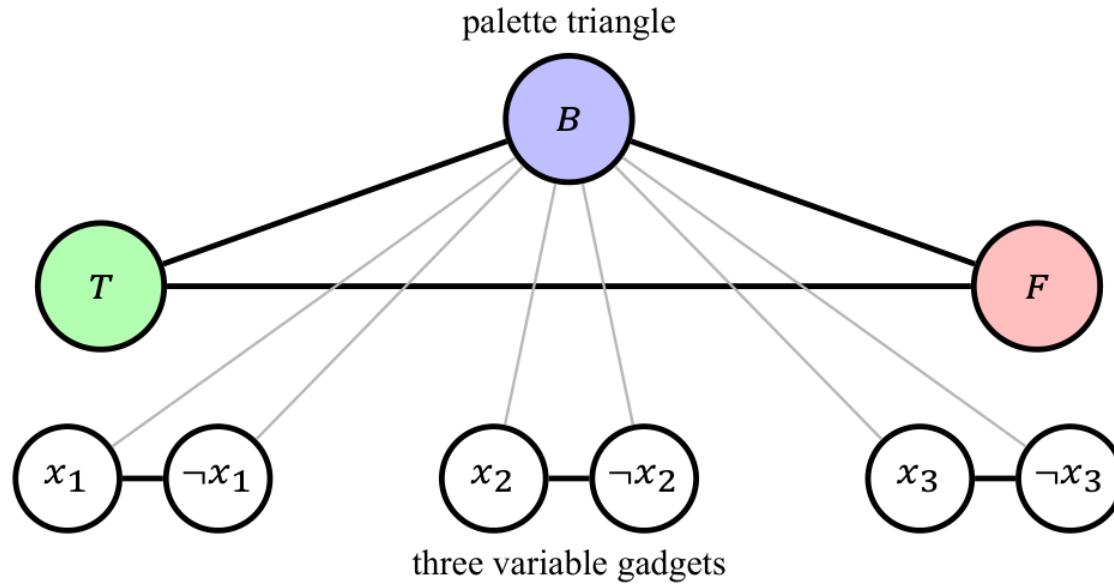
$$C_1 = (T, F, F), \quad C_2 = (F, T, T).$$

Both clause gadgets can be extended.

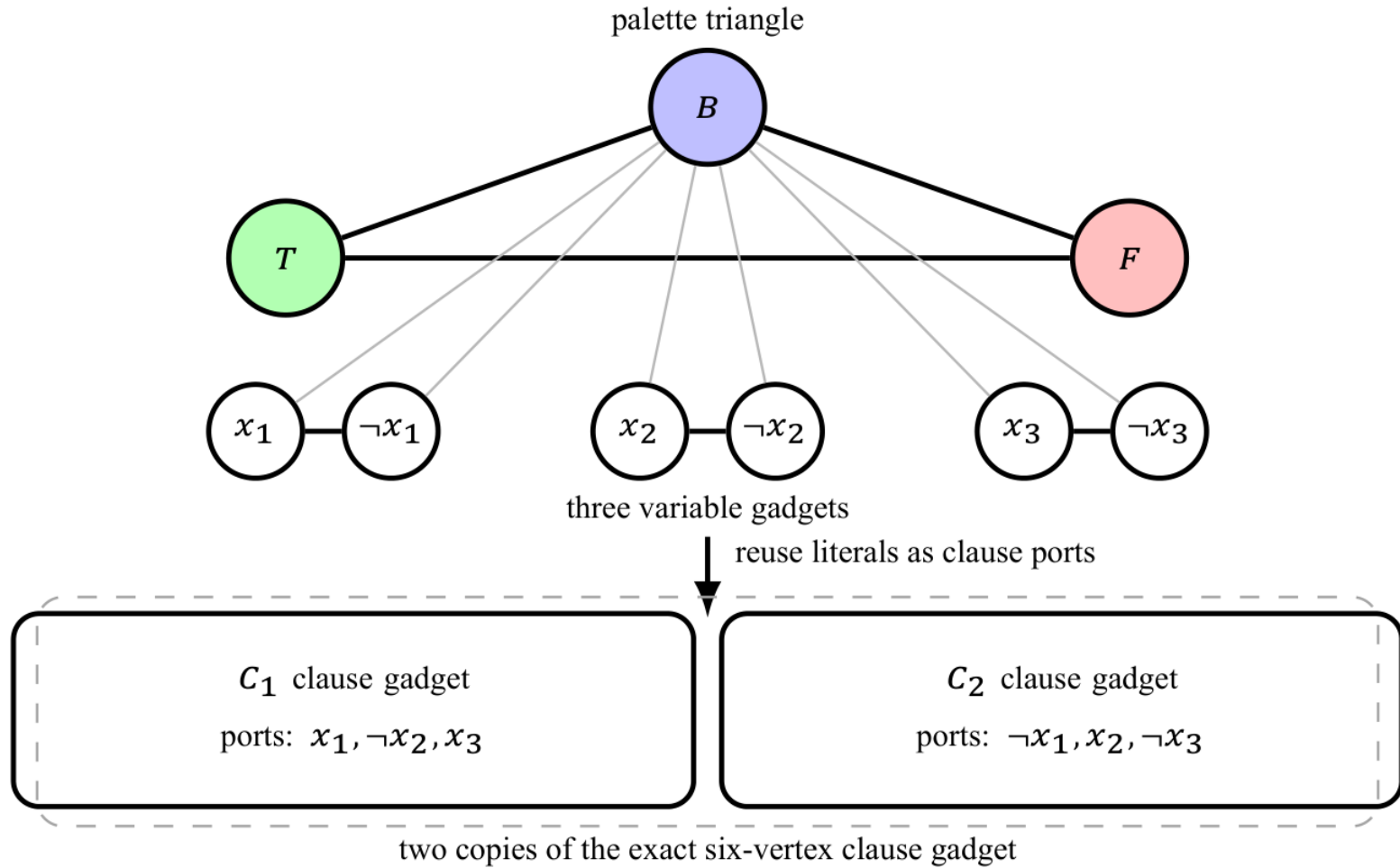
Assembling the Formula Graph



Assembling the Formula Graph



Assembling the Formula Graph



The palette and literal vertices are shared; each clause has fresh internal vertices.

Correctness and Complexity of the Reduction

ϕ satisfiable $\Rightarrow G_\phi$ 3-colorable

Give each literal its truth color.

Every clause has a true literal.

Every clause gadget therefore extends to a legal coloring.

Correctness and Complexity of the Reduction

ϕ satisfiable $\Rightarrow G_\phi$ 3-colorable

Give each literal its truth color.

Every clause has a true literal.

Every clause gadget therefore extends to a legal coloring.

G_ϕ 3-colorable $\Rightarrow \phi$ satisfiable

Read $x_i = \text{true}$ exactly when vertex x_i has color T .

Every colorable clause has a T input.

Every clause is therefore satisfied.

Correctness and Complexity of the Reduction

ϕ satisfiable $\Rightarrow G_\phi$ 3-colorable

Give each literal its truth color.

Every clause has a true literal.

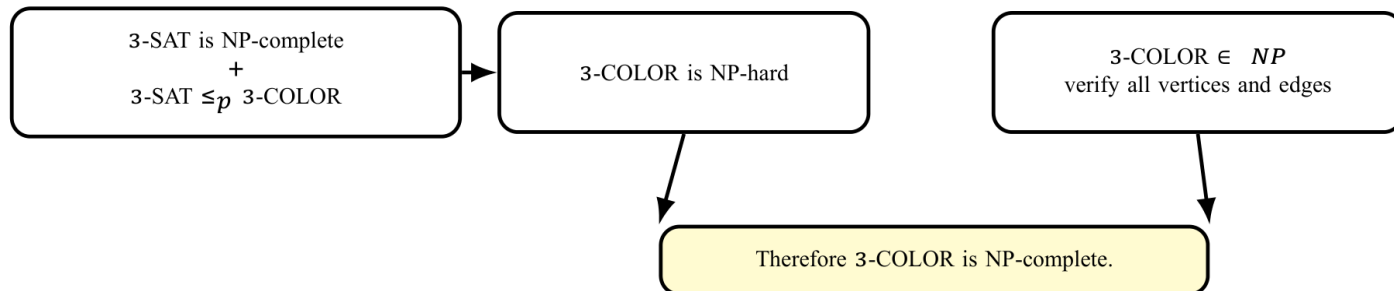
Every clause gadget therefore extends to a legal coloring.

G_ϕ 3-colorable $\Rightarrow \phi$ satisfiable

Read $x_i = \text{true}$ exactly when vertex x_i has color T .

Every colorable clause has a T input.

Every clause is therefore satisfied.



For n variables and m clauses:

$$|V| = 3 + 2n + 6m, \quad |E| = 3 + 3n + 13m.$$

Enumerating the vertices and edges takes polynomial time.

A Second Reduction Pattern: 3-SAT reduces to CLIQUE

$$\begin{aligned}\phi' = & (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \\ & \wedge (x_1 \vee x_2 \vee \neg x_3).\end{aligned}$$

Create one vertex per literal occurrence.

Put vertices in clause groups.

Connect different-clause vertices when their literals are not contradictory.

Ask for m pairwise adjacent vertices: an m -clique.

A Second Reduction Pattern: 3-SAT reduces to CLIQUE

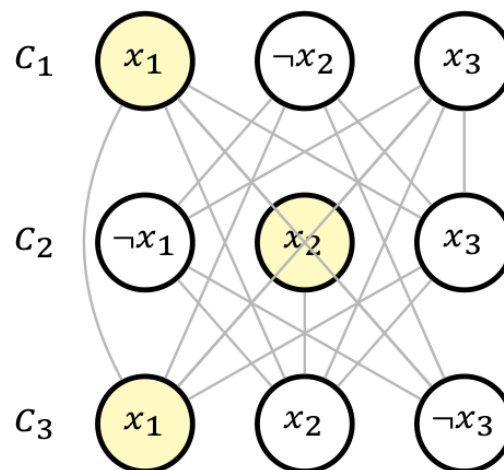
$$\phi' = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3).$$

Create one vertex per literal occurrence.

Put vertices in clause groups.

Connect different-clause vertices when their literals are not contradictory.

Ask for m pairwise adjacent vertices: an m -clique.



A Second Reduction Pattern: 3-SAT reduces to CLIQUE

$$\phi' = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \\ \wedge (x_1 \vee x_2 \vee \neg x_3).$$

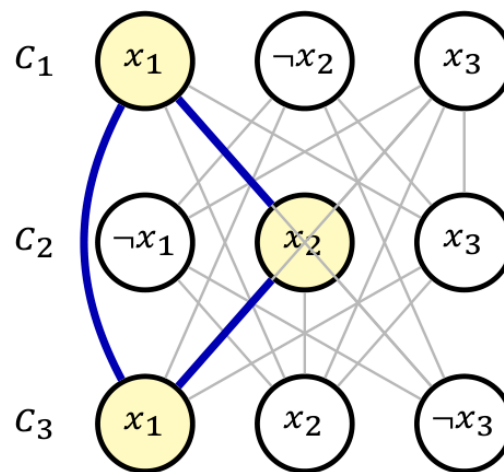
Create one vertex per literal occurrence.

Put vertices in clause groups.

Connect different-clause vertices when their literals are not contradictory.

Ask for m pairwise adjacent vertices: an m -clique.

One pairwise noncontradictory literal occurrence from each clause forms a clique.



Conversely, an m -clique chooses one occurrence per clause with no complementary pair. Set them true.

Construction: $3m$ vertices and $O(m^2)$ edges.

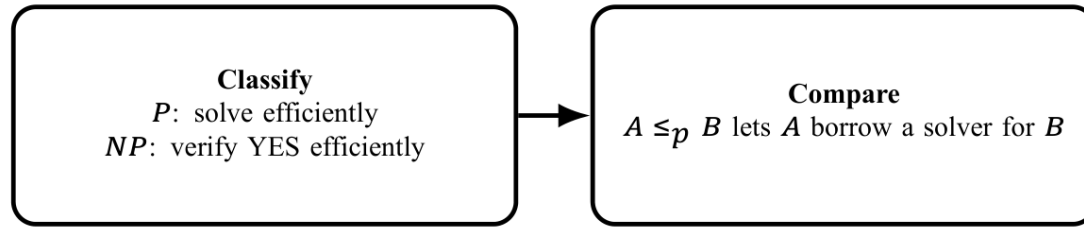
Takeaway

Classify

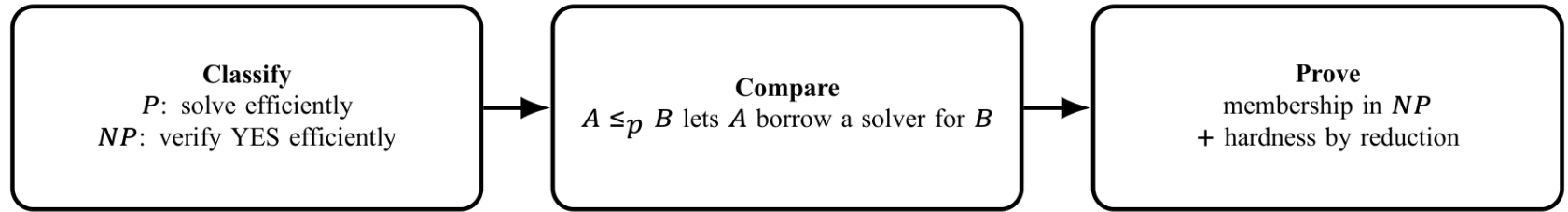
P: solve efficiently

NP: verify YES efficiently

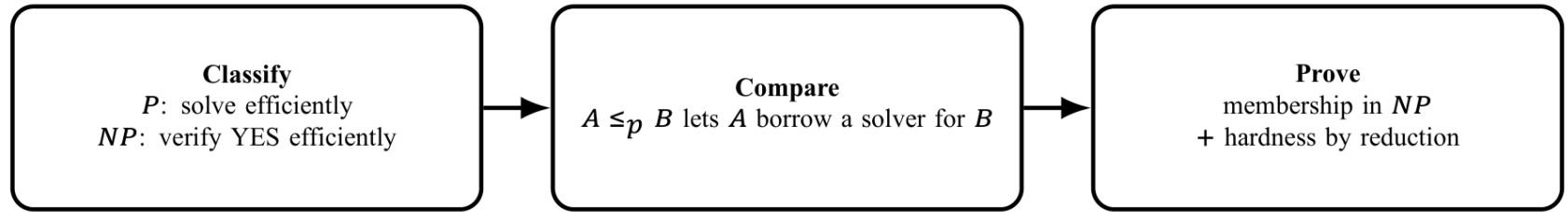
Takeaway



Takeaway



Takeaway



One reusable sentence

To prove a target B NP-complete, show $B \in NP$ and reduce a known NP-complete problem A to B in polynomial time while preserving YES/NO answers.

Questions?

Core references: Sipser; Cormen–Leiserson–Rivest–Stein; Princeton COS 423 reduction notes.

References

Primary references

Michael Sipser, *Introduction to the Theory of Computation*.

Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*.

CMU 15-451/651 lecture notes on NP-completeness.

Princeton COS 423 lecture notes, *Polynomial-Time Reductions*.

The exact six-vertex clause gadget and its 13-edge construction follow the

[Princeton reduction notes](#).